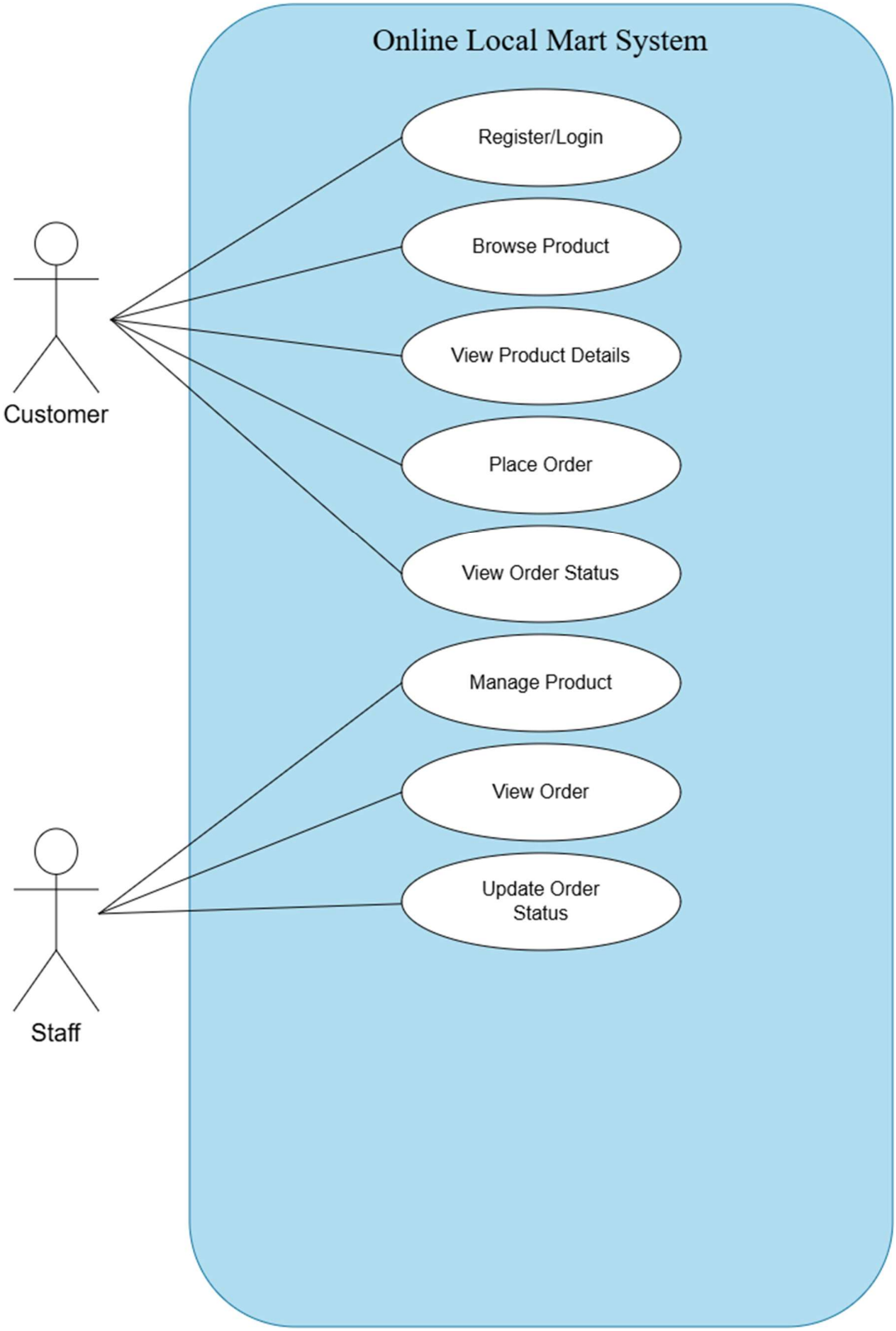
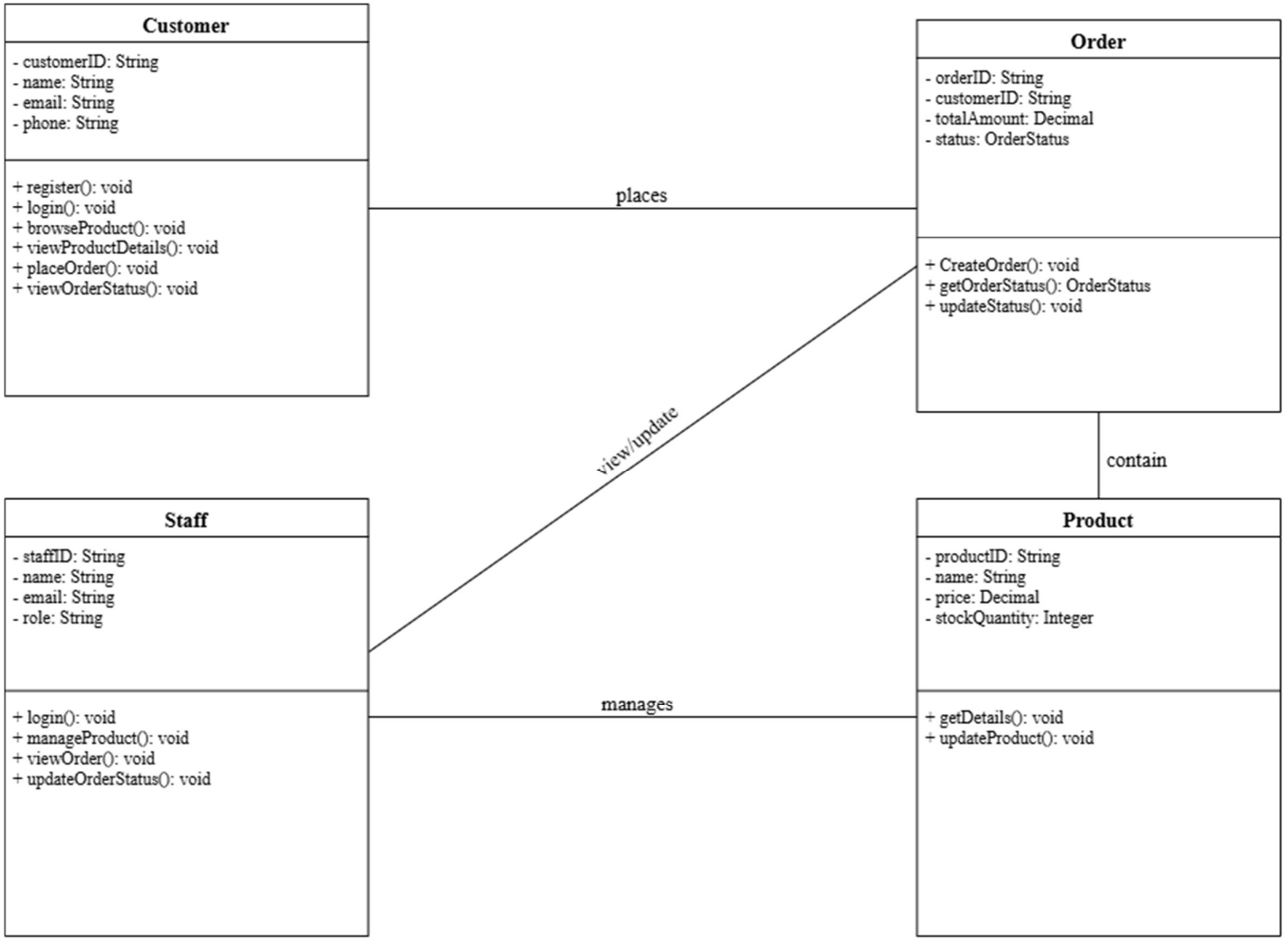
Activity 1: Use Case Diagram



UML Class Diagram



Activity 3(a) Naming Conventions – Dictionary

1. General Architectural Casing Conventions

Element	Notation Style	Standard / Syntax Rule	OLMS Implementation Example
Classes / Entities	`PascalCase`	Nouns representing primary domain entities matching the UML class definitions.	`Customer`, `Staff`, `Product`, `Order`
Methods / Functions	`camelCase`	Verbs or verb-object phrases defining operations and behaviors.	`browseProduct()`, `viewOrderStatus()`, `updateProduct()`
Attributes / Variables	`camelCase`	Descriptive nouns or compound identifiers specifying state.	`customerID`, `stockQuantity`, `totalAmount`
Constants / Enums	`UPPER_SNAKE_CASE`	Immutable state definitions and enumerations.	`OrderStatus.PENDING`, `OrderStatus.DELIVERED`
Database Tables	`snake_case` (Plural)	Plural nouns representing persistent relational tables.	`customers`, `staffs`, `products`, `orders`
Database Columns	`snake_case` (Singular)	Lowercase identifiers; primary and foreign keys suffixed with `_id`.	`customer_id`, `product_id`, `stock_quantity`
Source Code Files	`PascalCase`	Filenames must strictly mirror the class name they encapsulate.	`Customer.php` / `Customer.js`, `Product.php`

2. Domain Entities & Attributes Dictionary

Directly derived from the OLMS UML specification to ensure uniform variable naming across UI forms, backend logic, and database schemas:

Entity	Attribute Name (Code)	Database Column	Data Type	Description
Customer	`customerID`	`customer_id`	`String/VARCHAR(50)`	Unique primary identifier for a customer account.
	`name`	`name`	`String/VARCHAR(100)`	Full legal name of the customer.
	`email`	`email`	`String/VARCHAR(100)`	Registered customer email address (unique).
	`phone`	`phone`	`String/VARCHAR(20)`	Contact phone number for delivery coordination.
Staff	`staffID`	`staff_id`	`String/VARCHAR(50)`	Unique primary identifier for staff/mart admin.
	`name`	`name`	`String/VARCHAR(100)`	Full name of the staff member.
	`email`	`email`	`String/VARCHAR(100)`	Official mart personnel email address.
	`role`	`role`	`String/VARCHAR(50)`	Access role assigned (e.g., `Manager`, `Clerk`).

Entity	Attribute Name (Code)	Database Column	Data Type	Description
Product	`productID`	`product_id`	`String / VARCHAR(50)`	Unique inventory SKU / product identifier.
	`name`	`name`	`String / VARCHAR(150)`	Commercial name of the listed product.
	`price`	`price`	`Decimal(10,2)`	Unit selling price of the product.
	`stockQuantity`	`stock_quantity`	`Integer / INT`	Current units available in local mart inventory.
Order	`orderID`	`order_id`	`String / VARCHAR(50)`	Unique invoice / order tracking code.
	`customerID`	`customer_id`	`String / VARCHAR(50)`	Foreign key referencing `Customer.customerID`.
	`totalAmount`	`total_amount`	`Decimal(10,2)`	Final calculated financial cost of the order.
	`status`	`status`	`OrderStatus` (Enum / `VARCHAR`)	Current delivery/processing status of the order.

3. Method & Function Dictionary

Derived directly from the Use Case actions and UML Class operations:

Method Signature	Class Origin	Access	Return Type	Description / Responsibility
`register()`	`Customer`	`public (+)`	`void`	Creates and validates a new customer account.
`login()`	`Customer, Staff`	`public (+)`	`void`	Authenticates user credentials and initiates session.
`browseProduct()`	`Customer`	`public (+)`	`void`	Queries and displays active products to the customer.
`viewProductDetails()`	`Customer`	`public (+)`	`void`	Displays granular specifications, stock, and price for a single product.
`placeOrder()`	`Customer`	`public (+)`	`void`	Converts selected items into a pending customer order.
`viewOrderStatus()`	`Customer`	`public (+)`	`void`	Fetches the current fulfillment state of customer orders.
`manageProduct()`	`Staff`	`public (+)`	`void`	Coordinates CRUD operations over the product inventory.
`viewOrder()`	`Staff`	`public (+)`	`void`	Retrieves orders across the mart for processing.

Method Signature	Class Origin	Access	Return Type	Description / Responsibility
`updateOrderStatus()`	`Staff`, `Order`	`public` (+)	`void`	Mutates order lifecycle (e.g., Pending $\rightarrow$ Shipped $\rightarrow$ Delivered).
`getDetails()`	`Product`	`public` (+)	`void`	Returns internal product properties for viewing.
`updateProduct()`	`Product`	`public` (+)	`void`	Mutates existing product price, name, or stock quantity.
`createOrder()`	`Order`	`public` (+)	`void`	Instantiates and persists a new order record.
`getOrderStatus()`	`Order`	`public` (+)	`OrderStatus`	Queries the current enum value of the order status.

4. Prohibited Terminology & Synonyms Rule

To prevent inconsistencies across team pull requests, only the official terms from the diagram may be used:

- Use `Customer`, never `User`, `Buyer`, or `Client`.
- Use `Staff`, never `Admin`, `Merchant`, `Employee`, or `Seller`.
- Use `Product`, never `Item`, `Good`, or `Merchandise`.
- Use `stockQuantity`, never `qty`, `count`, `stock`, or `amount`.
- Use `totalAmount`, never `total`, `price`, `cost`, or `bill`.

Activity 3(b): Define Functions and Modularity

Function and Method Modularity

Customer Class

- register() creates a new account for the customer with information such as name, email and phone number.
- login() verifies customer's login details before allowing access to the system.
- browseProduct() Search and display any available products for the customer.
- viewProductDetails() shows detailed information for a selected product, such as product name, price, and stock quantity.
- placeOrder() creates an order after the customer selects products to purchase.
- viewOrderStatus() displays the current status of the customer's order, such as Pending,
- Processing, Shipped, or Completed.

Staff Class

- login() verifies staff login credentials.
- manageProduct() coordinates product management tasks, such as adding, editing, or removing a product.
- viewOrder() retrieves and displays customer order information for staff.
- updateOrderStatus() changes the status of an order after it has been processed.
- Product Class
- getDetails() returns the details of a product for customers or staff to view.
- updateProduct() updates product information, such as the product name, price, or stock quantity.

Order Class

- CreateOrder() creates a new order when a customer places an order.
- getOrderStatus() returns the current order status for the customer or staff.
- updateStatus() changes the order status when staff process the order.

Activity 4: Define Communication and Continuous Integration Rules

a. Communication Rules

Category	Policy / Standard	Operational Details
Communication Channels	Centralized Platform	All project-related communication should be conducted through the agreed team communication platform.
Task Management	Issue Tracking via GitHub	GitHub Issues should be used to report bugs, assign development tasks, and track project-related problems.
Defect Reporting	Structured Issue Descriptions	Team members should clearly describe the issue, expected outcome, and relevant module when creating an issue.
Progress Reporting	Transparency & Updates	Team members should provide regular updates on their assigned tasks and inform the team early about any problems or delays.
System Decisions	Prior Technical Consensus	Technical decisions that affect the system architecture, database, coding standards, or development workflow should be discussed with the team before implementation.
Pull Request Content	Clear Context & Traceability	Pull Requests should include a clear description of the changes made and the related issue or task where applicable.
Code Review	Mandatory Peer Review	Team members should review and respond to Pull Request comments before the code is merged.
Knowledge Base	GitHub Documentation	Important project decisions and changes should be documented in GitHub so that they can be referred to by all team members.
Domain Standards	Dictionary Alignment	Team members should use the approved domain terminology consistently, such as `Customer`, `Staff`, `Product`, `Order`, `stockQuantity`, and `totalAmount`.

Category	Policy / Standard	Operational Details
Workflow Governance	Standard Alterations	Any changes to the agreed coding standards, system structure, or development workflow should be communicated to the entire team.

b. Continuous Integration Rules

Pipeline Stage	CI Standard	Enforcement Criteria
Branching Strategy	Isolated Feature Branches	All new features and fixes must be developed using a separate `feature` branch.
Branch Protection	Protected Primary Branches	Developers must not directly push changes to the `main` or `dev` branches.
Staging Workflow	Targeted Pull Requests	Completed features must be submitted through a Pull Request to the `dev` branch.
Review Gate	Minimum Approvals	Each Pull Request must be reviewed by at least one other team member before merging.
Automation Trigger	Automated CI Runs	CI checks must run automatically when code is pushed to a feature branch or when a Pull Request is created or updated.
Build & Test	Integrity Verification	CI must verify that the project can be built successfully and that automated tests, where available, pass successfully.
Quality Gate	Zero-Failure Policy	Code that fails the required CI checks must not be merged until the errors are fixed.
Integration	Staging Verification	The `dev` branch is used to integrate, and test completed features before they are released to `main`.
Release Management	Stable Main Branch	The `main` branch must contain only stable and production-ready code.
Regression Prevention	Core Module Safeguards	Changes merged into `dev` should not break existing system modules such as Customer, Staff, Product, and Order functions.
Coding Standards	Pre-PR Compliance	Developers should ensure that their code follows the approved naming conventions and modularity rules before creating a Pull Request.
Code Modularity	Function Length Limit	Functions should remain within the agreed maximum of 30 lines unless there is a documented reason for exceeding the limit.

Pipeline Stage	CI Standard	Enforcement Criteria
Defect Ownership	Responsible Triage	Any failed CI build or test should be investigated and resolved by the developer responsible for the change.

Lab 3 & 4 and 5 & 6 : Constructions of Input and Output

Class Design (Attributes and Methods)

1) Class Product

```
class Product {
    #productID; #name; #price; #stockQuantity;
    constructor(productID, name, price, stockQuantity) {
        this.#productID = String(productID).trim();
        this.#name = String(name).trim();
        this.#price = parseFloat(price);
        this.#stockQuantity = parseInt(stockQuantity, 10);
    }
    get productID() { return this.#productID; }
    get name() { return this.#name; }
    get price() { return this.#price; }
    get stockQuantity() { return this.#stockQuantity; }
    set name(value) { if (!value || value.trim() === "") throw new Error(
        "Product name cannot be empty."); this.#name = value.trim(); }
    set price(value) { const parsed = parseFloat(value); if (isNaN(parsed
        ) || parsed < 0) throw new Error("Price must be a non-negative
        decimal."); this.#price = parsed; }
    set stockQuantity(value) { const parsed = parseInt(value, 10); if (
        isNaN(parsed) || parsed < 0) throw new Error("Stock quantity
        must be a positive integer."); this.#stockQuantity = parsed; }
    getDetails() { return { productID: this.#productID, name: this.#name,
         price: this.#price, stockQuantity: this.#stockQuantity }; }
    updateProduct(name, price, stockQuantity) { this.name = name; this.
        price = price; this.stockQuantity = stockQuantity; }
}
```

2) Class ProductRepository

```
class ProductRepository {
    #storage = [];
    constructor() {
        this.create(new Product("P001", "Fresh Farm Milk 1L", 3.50, 25));
        this.create(new Product("P002", "Whole Wheat Bread", 2.80, 18));
        this.create(new Product("P003", "Organic Brown Eggs (10s)", 4.20,
            30));
        this.create(new Product("P004", "Green Gala Apples (1kg)", 5.90,
            12));
    }
    getAll() { return this.#storage; }
    findById(productID) { const query = String(productID).trim().
        toLowerCase(); return this.#storage.find(p => p.productID.
        toLowerCase() === query) || null; }
    findAny(query) { const q = String(query).trim().toLowerCase(); return
         this.#storage.filter(p => p.productID.toLowerCase().includes(q)
        || p.name.toLowerCase().includes(q)); }
    create(product) { if (this.findById(product.productID)) { throw new
        Error(`Product ID ${product.productID} already exists in
        database.`); } this.#storage.push(product); return product; }
    update(productID, name, price, stockQuantity) { const product = this.
        findById(productID); if (!product) return null; product.
        updateProduct(name, price, stockQuantity); return product; }
    delete(productID) { const index = this.#storage.findIndex(p => p.
        productID.toLowerCase() === productID.toLowerCase()); if (index
        === -1) return false; this.#storage.splice(index, 1); return true
        ; }
}
```

3) Class Order

```
class Order {
    #orderID; #customerID; #totalAmount; #status;
    constructor(orderID, customerID, totalAmount, status) {
        this.#orderID = String(orderID).trim();
        this.#customerID = String(customerID).trim();
        this.#totalAmount = parseFloat(totalAmount);
        this.#status = String(status).trim().toUpperCase();
    }
    get orderID() { return this.#orderID; }
    get customerID() { return this.#customerID; }
    get totalAmount() { return this.#totalAmount; }
    get status() { return this.#status; }
    set totalAmount(value) { const parsed = parseFloat(value); if (isNaN(
        parsed) || parsed < 0) throw new Error("Total amount must be
        non-negative."); this.#totalAmount = parsed; }
    set status(value) { const valid = Object.values(OrderStatus); if (!
        valid.includes(value.toUpperCase())) throw new Error("Invalid
        order status."); this.#status = value.toUpperCase(); }
    getDetails() { return { orderID: this.#orderID, customerID: this.#
        customerID, totalAmount: this.#totalAmount, status: this.#status
        }; }
    updateOrder(customerID, totalAmount, status) { this.#customerID =
        customerID; this.totalAmount = totalAmount; this.status = status;
         }
}
```

4) Class OrderRepository

```
class OrderRepository {
    #storage = [];
    constructor() {
        this.create(new Order("O001", "C001", 15.50, OrderStatus.PENDING)
            );
        this.create(new Order("O002", "C002", 22.00, OrderStatus.
            COMPLETED));
    }
    getAll() { return this.#storage; }
    findById(orderID) { const query = String(orderID).trim().toLowerCase(
        ); return this.#storage.find(o => o.orderID.toLowerCase() ===
        query) || null; }
    findAny(query) { const q = String(query).trim().toLowerCase(); return
         this.#storage.filter(o => o.orderID.toLowerCase().includes(q) ||
         o.customerID.toLowerCase().includes(q)); }
    create(order) { if (this.findById(order.orderID)) { throw new Error(`
        Order ID ${order.orderID} already exists.`); } if (!custRepo.
        findById(order.customerID)) { throw new Error(`Customer ID ${
        order.customerID} does not exist.`); } this.#storage.push(order);
         return order; }
    update(orderID, customerID, totalAmount, status) { const order = this
        .findById(orderID); if (!order) return null; order.updateOrder(
        customerID, totalAmount, status); return order; }
    delete(orderID) { const index = this.#storage.findIndex(o => o.
        orderID.toLowerCase() === orderID.toLowerCase()); if (index === -
        1) return false; this.#storage.splice(index, 1); return true; }
}
```

5) Class Staff

```
class Staff {
    #staffID; #name; #email; #role;
    constructor(staffID, name, email, role) {
        this.#staffID = String(staffID).trim();
        this.#name = String(name).trim();
        this.#email = String(email).trim();
        this.#role = String(role).trim();
    }
    get staffID() { return this.#staffID; }
    get name() { return this.#name; }
    get email() { return this.#email; }
    get role() { return this.#role; }
    set name(value) { if (!value || value.trim() === "") throw new Error(
        "Staff name cannot be empty."); this.#name = value.trim(); }
    set email(value) { if (!value.includes('@')) throw new Error("
        Invalid email format."); this.#email = value.trim(); }
    set role(value) { if (!value || value.trim() === "") throw new Error(
        "Role cannot be empty."); this.#role = value.trim(); }
    getDetails() { return { staffID: this.#staffID, name: this.#name,
        email: this.#email, role: this.#role }; }
    updateStaff(name, email, role) { this.name = name; this.email = email
        ; this.role = role; }
}
```

6) Class StaffRepository

```
class StaffRepository {
    #storage = [];
    constructor() {
        this.create(new Staff("S001", "Admin User", "admin@olms.com", "
            Manager"));
    }
    getAll() { return this.#storage; }
    findById(staffID) { const query = String(staffID).trim().toLowerCase(
        ); return this.#storage.find(s => s.staffID.toLowerCase() ===
        query) || null; }
    findAny(query) { const q = String(query).trim().toLowerCase(); return
         this.#storage.filter(s => s.staffID.toLowerCase().includes(q) ||
         s.name.toLowerCase().includes(q)); }
    create(staff) { if (this.findById(staff.staffID)) { throw new Error(`
        Staff ID ${staff.staffID} already exists.`); } this.#storage.push
        (staff); return staff; }
    update(staffID, name, email, role) { const staff = this.findById(
        staffID); if (!staff) return null; staff.updateStaff(name, email,
         role); return staff; }
    delete(staffID) { const index = this.#storage.findIndex(s => s.
        staffID.toLowerCase() === staffID.toLowerCase()); if (index === -
        1) return false; this.#storage.splice(index, 1); return true; }
}
```

CRUD Functions in Each Module

1) Product Module

a) Create: handleCreateProduct()

```
function handleCreateProduct(event) {
    event.preventDefault();
    const feedback = document.getElementById("createFeedback");
    try {
        const id = document.getElementById("create_productID").value;
        const name = document.getElementById("create_name").value;
        const price = document.getElementById("create_price").value;
        const stock = document.getElementById("create_stockQuantity").
            value;
        const newProd = new Product(id, name, price, stock);
        repo.create(newProd);
        feedback.className = "message-box msg-success";
        feedback.textContent = `Product [${newProd.productID}] "${newProd
            .name}" created successfully.`;
        document.getElementById("createProductForm").reset();
        renderInventoryTable();
    } catch (err) { feedback.className = "message-box msg-error";
        feedback.textContent = err.message; }
}
```

b) Read: handleSearchProduct()

```
function handleSearchProduct(event) {
    event.preventDefault();
    const query = document.getElementById("search_query").value;
    const feedback = document.getElementById("searchFeedback");
    const resultsContainer = document.getElementById("
        searchResultsDisplay");
    resultsContainer.innerHTML = "";
    const matches = repo.findAny(query);
    if (matches.length === 0) { feedback.className = "message-box
        msg-error"; feedback.textContent = `Not found: No product
        matching "${query}".`; return; }
    feedback.className = "message-box msg-info";
    feedback.textContent = `Search successful: Found ${matches.length}
        matching product(s).`;
    let html = `<table><thead><tr><th>ID</th><th>Name</th><th>
        Price</th><th>Stock</th><th>Action</th></tr></thead><tbody>`;
    matches.forEach(p => { html += `<tr><td>${p.productID}</td><td>${p.
        name}</td><td>$${p.price.toFixed(2)}</td><td>${p.stockQuantity}
        </td><td><button class="btn-action" onclick="openEditWorkspace('
        ${p.productID}')">Select</button></td></tr>`; });
    html += `</tbody></table>`;
    resultsContainer.innerHTML = html;
}
```

c) Update: handleSaveEdit()

```
function handleSaveEdit(event) {
    event.preventDefault();
    const id = document.getElementById("edit_productID").value;
    const name = document.getElementById("edit_name").value;
    const price = document.getElementById("edit_price").value;
    const stock = document.getElementById("edit_stockQuantity").value;
    const feedback = document.getElementById("editFeedback");
    try {
        const updated = repo.update(id, name, price, stock);
        feedback.className = "message-box msg-success";
        feedback.textContent = `Product ${updated.productID} updated
            successfully. Validation verified.`;
        renderInventoryTable();
    } catch (err) { feedback.className = "message-box msg-error";
        feedback.textContent = err.message; }
}
```

d) Delete: handleDeletePrompt()

```
function handleDeletePrompt() {
    const id = document.getElementById("edit_productID").value;
    const product = repo.findById(id);
    if (!product) return;
    const confirmed = confirm(`Confirm Deletion?\n\nProduct ID: ${product
        .productID}\nName: ${product.name}\nPrice: $${product.price.
        toFixed(2)}\nStock: ${product.stockQuantity}\n\nAre you sure you
        want to delete this record?`);
    if (confirmed) { repo.delete(id); closeEditWorkspace();
        renderInventoryTable(); alert(`Product ${id} has been
        permanently deleted.`); }
}
```

2) Customer Module

a) Create: handleCreateCustomer

```
function handleCreateCustomer(event) {
    event.preventDefault();
    const fb = document.getElementById("c_createFeedback");
    try {
        const id = document.getElementById("c_create_id").value;
        const name = document.getElementById("c_create_name").value;
        const email = document.getElementById("c_create_email").value;
        const phone = document.getElementById("c_create_phone").value;
        const newC = new Customer(id, name, email, phone);
        custRepo.create(newC);
        fb.className = "message-box msg-success";
        fb.textContent = `Customer [${newC.customerID}] registered
            successfully.`;
        document.getElementById("createCustomerForm").reset();
        renderCustomerTable();
    } catch (err) { fb.className = "message-box msg-error"; fb.
        textContent = err.message; }
}
```

b) Read: handleSearchCustomer()

```
function handleSearchCustomer(event) {
    event.preventDefault();
    const q = document.getElementById("c_search_query").value;
    const fb = document.getElementById("c_searchFeedback");
    const res = document.getElementById("c_searchResultsDisplay");
    res.innerHTML = "";
    const matches = custRepo.findAny(q);
    if (matches.length === 0) { fb.className = "message-box msg-error";
        fb.textContent = `Not found: "${q}".`; return; }
    fb.className = "message-box msg-info";
    fb.textContent = `Found ${matches.length} customer(s).`;
    let html = `<table><thead><tr><th>ID</th><th>Name</th><th>
        Email</th><th>Phone</th><th>Action</th></tr></thead><tbody>`;
    matches.forEach(c => { html += `<tr><td>${c.customerID}</td><td>${c.
        name}</td><td>${c.email}</td><td>${c.phone}</td><td><button
        class="btn-action" onclick="openCustomerEdit('${c.customerID}
        ')">Select</button></td></tr>`; });
    res.innerHTML = html + `</tbody></table>`;
}
```

c) Update: handleSaveCustomerEdit()

```
function handleSaveCustomerEdit(event) {
    event.preventDefault();
    const id = document.getElementById("c_edit_id").value;
    const name = document.getElementById("c_edit_name").value;
    const email = document.getElementById("c_edit_email").value;
    const phone = document.getElementById("c_edit_phone").value;
    const fb = document.getElementById("c_editFeedback");
    try {
        const u = custRepo.update(id, name, email, phone);
        fb.className = "message-box msg-success";
        fb.textContent = `Customer ${u.customerID} updated.`;
        renderCustomerTable();
    } catch (err) { fb.className = "message-box msg-error"; fb.
        textContent = err.message; }
}
```

d) Delete: handleDeleteCustomerPrompt()

```
function handleDeleteCustomerPrompt() {
    const id = document.getElementById("c_edit_id").value;
    const c = custRepo.findById(id);
    if (!c) return;
    if (confirm(`Delete Customer ${c.customerID} (${c.name})?`)) {
        custRepo.delete(id); closeCustomerWorkspace();
        renderCustomerTable(); alert(`Deleted ${id}.`); }
}
```

3) Order Module

a) Create: handleCreateOrder()

```
function handleCreateOrder(event) {
    event.preventDefault();
    const fb = document.getElementById("o_createFeedback");
    try {
        const id = document.getElementById("o_create_id").value;
        const cid = document.getElementById("o_create_cid").value;
        const amt = document.getElementById("o_create_amount").value;
        const st = document.getElementById("o_create_status").value;
        const newO = new Order(id, cid, amt, st);
        orderRepo.create(newO);
        fb.className = "message-box msg-success";
        fb.textContent = `Order [${newO.orderID}] created successfully.`;
        document.getElementById("createOrderForm").reset();
        renderOrderTable();
    } catch (err) { fb.className = "message-box msg-error"; fb.
        textContent = err.message; }
}
```

b) Read: handleSearchOrder()

```
function handleSearchOrder(event) {
    event.preventDefault();
    const q = document.getElementById("o_search_query").value;
    const fb = document.getElementById("o_searchFeedback");
    const res = document.getElementById("o_searchResultsDisplay");
    res.innerHTML = "";
    const matches = orderRepo.findAny(q);
    if (matches.length === 0) { fb.className = "message-box msg-error";
        fb.textContent = `Not found: "${q}".`; return; }
    fb.className = "message-box msg-info";
    fb.textContent = `Found ${matches.length} order(s).`;
    let html = `<table><thead><tr><th>ID</th><th>Cust ID</th><th>
        Amount</th><th>Status</th><th>Action</th></tr></thead><tbody>`;
    matches.forEach(o => { html += `<tr><td>${o.orderID}</td><td>${o.
        customerID}</td><td>$${o.totalAmount.toFixed(2)}</td><td>${o.
        status}</td><td><button class="btn-action"
        onclick="openOrderEdit('${o.orderID}')">Select</button></td></tr>
        `; });
    res.innerHTML = html + `</tbody></table>`;
}
```

c) Update: handleSaveOrderEdit()

```
function handleSaveOrderEdit(event) {
    event.preventDefault();
    const id = document.getElementById("o_edit_id").value;
    const cid = document.getElementById("o_edit_cid").value;
    const amt = document.getElementById("o_edit_amount").value;
    const st = document.getElementById("o_edit_status").value;
    const fb = document.getElementById("o_editFeedback");
    try {
        const u = orderRepo.update(id, cid, amt, st);
        fb.className = "message-box msg-success";
        fb.textContent = `Order ${u.orderID} updated.`;
        renderOrderTable();
    } catch (err) { fb.className = "message-box msg-error"; fb.
        textContent = err.message; }
}
```

d) Delete: handleDeleteOrderPrompt()

```
function handleDeleteOrderPrompt() {
    const id = document.getElementById("o_edit_id").value;
    const o = orderRepo.findById(id);
    if (!o) return;
    if (confirm(`Delete Order ${o.orderID} ($${o.totalAmount.toFixed(2)}
        )?`)) { orderRepo.delete(id); closeOrderWorkspace();
        renderOrderTable(); alert(`Deleted ${id}.`); }
}
```

4) Staff Module

a) Create: handleCreateStaff()

```
function handleCreateStaff(event) {
    event.preventDefault();
    const fb = document.getElementById("s_createFeedback");
    try {
        const id = document.getElementById("s_create_id").value;
        const name = document.getElementById("s_create_name").value;
        const email = document.getElementById("s_create_email").value;
        const role = document.getElementById("s_create_role").value;
        const newS = new Staff(id, name, email, role);
        staffRepo.create(newS);
        fb.className = "message-box msg-success";
        fb.textContent = `Staff [${newS.staffID}] added successfully.`;
        document.getElementById("createStaffForm").reset();
        renderStaffTable();
    } catch (err) { fb.className = "message-box msg-error"; fb.
        textContent = err.message; }
}
```

b) Read: handleSearchStaff()

```
function handleSearchStaff(event) {
    event.preventDefault();
    const q = document.getElementById("s_search_query").value;
    const fb = document.getElementById("s_searchFeedback");
    const res = document.getElementById("s_searchResultsDisplay");
    res.innerHTML = "";
    const matches = staffRepo.findAny(q);
    if (matches.length === 0) { fb.className = "message-box msg-error";
        fb.textContent = `Not found: "${q}".`; return; }
    fb.className = "message-box msg-info";
    fb.textContent = `Found ${matches.length} staff member(s).`;
    let html = `<table><thead><tr><th>ID</th><th>Name</th><th>
        Email</th><th>Role</th><th>Action</th></tr></thead><tbody>`;
    matches.forEach(s => { html += `<tr><td>${s.staffID}</td><td>${s.name
        }</td><td>${s.email}</td><td>${s.role}</td><td><button
        class="btn-action" onclick="openStaffEdit('${s.staffID}
        ')">Select</button></td></tr>`; });
    res.innerHTML = html + `</tbody></table>`;
}
```

c) Update: handleSaveStaffEdit()

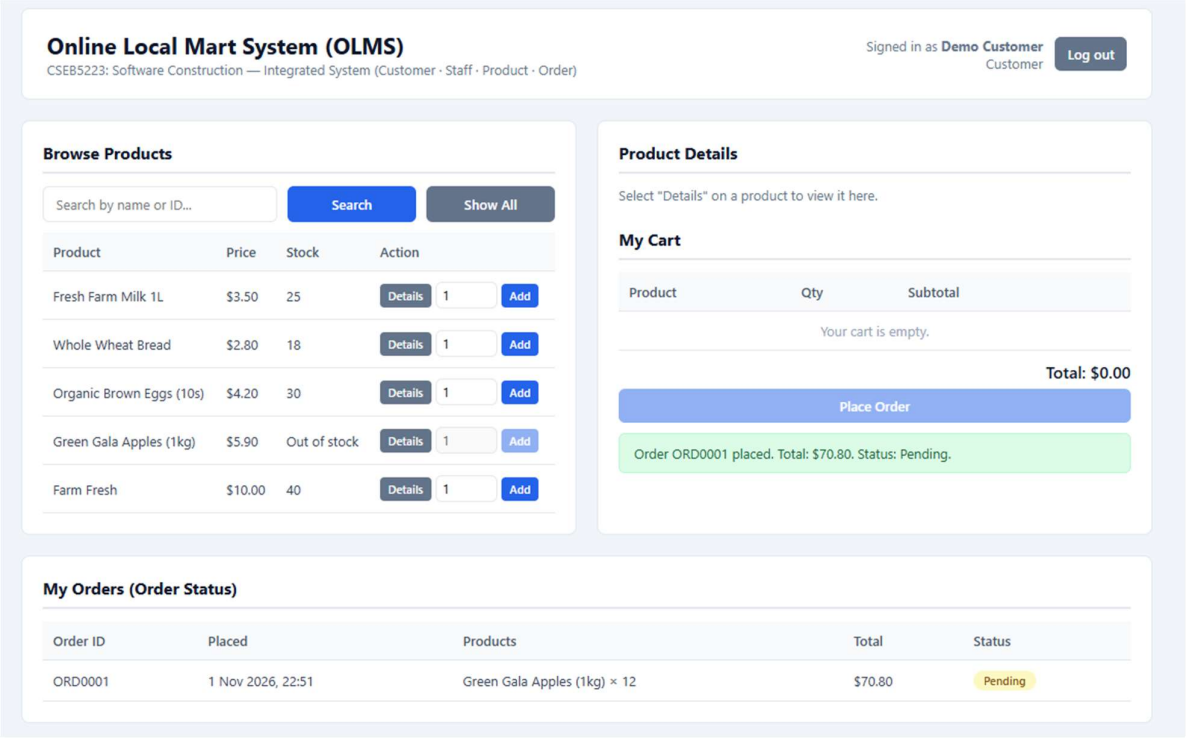
```
function handleSaveStaffEdit(event) {
    event.preventDefault();
    const id = document.getElementById("s_edit_id").value;
    const name = document.getElementById("s_edit_name").value;
    const email = document.getElementById("s_edit_email").value;
    const role = document.getElementById("s_edit_role").value;
    const fb = document.getElementById("s_editFeedback");
    try {
        const u = staffRepo.update(id, name, email, role);
        fb.className = "message-box msg-success";
        fb.textContent = `Staff ${u.staffID} updated.`;
        renderStaffTable();
    } catch (err) { fb.className = "message-box msg-error"; fb.
        textContent = err.message; }
}
```

d) Delete: handleDeleteStaffPrompt()

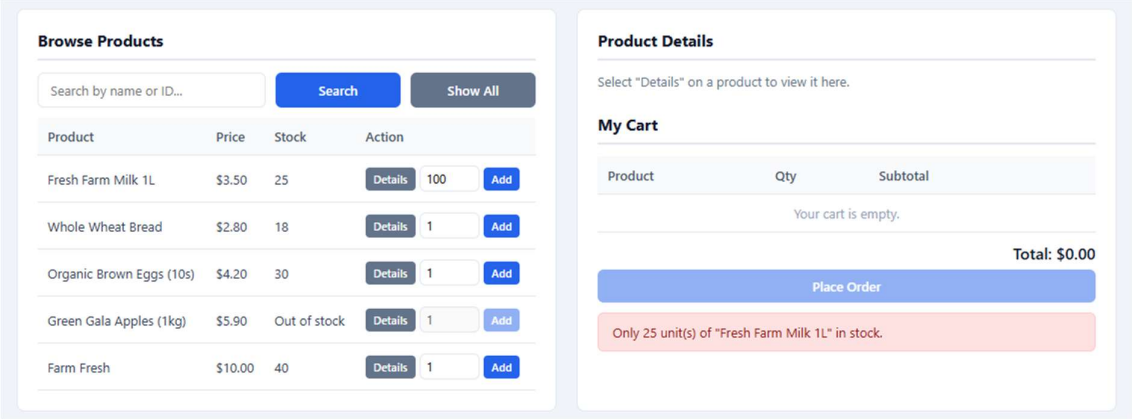
```
function handleDeleteStaffPrompt() {
    const id = document.getElementById("s_edit_id").value;
    const s = staffRepo.findById(id);
    if (!s) return;
    if (confirm(`Delete Staff ${s.staffID} (${s.name})?`)) { staffRepo.
        delete(id); closeStaffWorkspace(); renderStaffTable(); alert(`
        Deleted ${id}.`); }
}
```

Output:

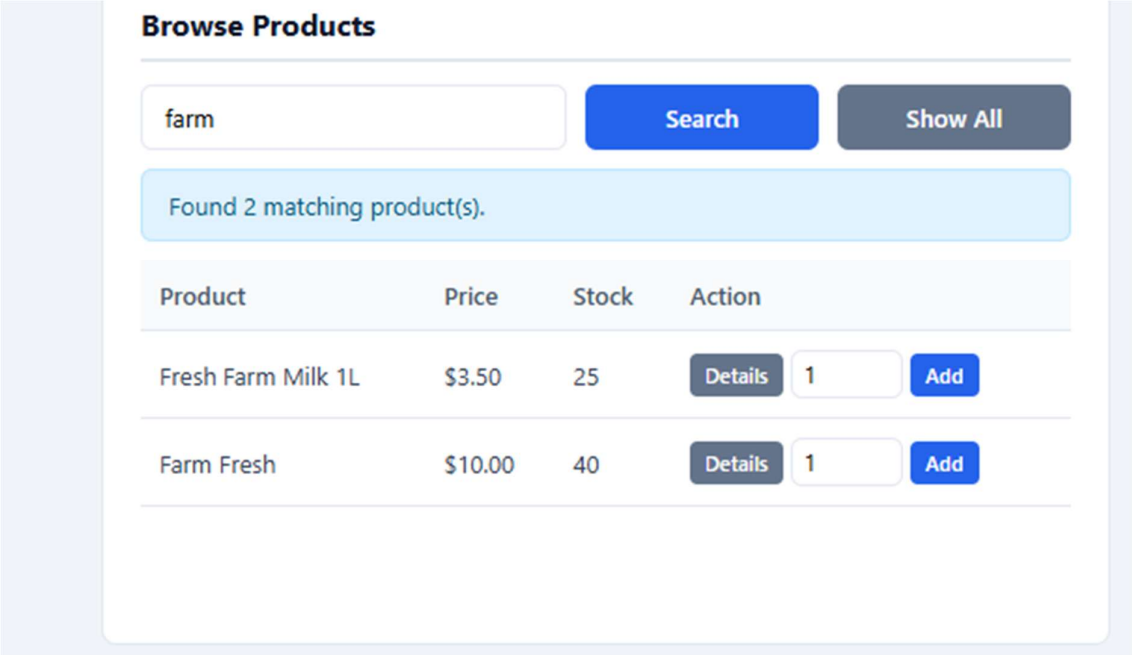
1. Customer Page

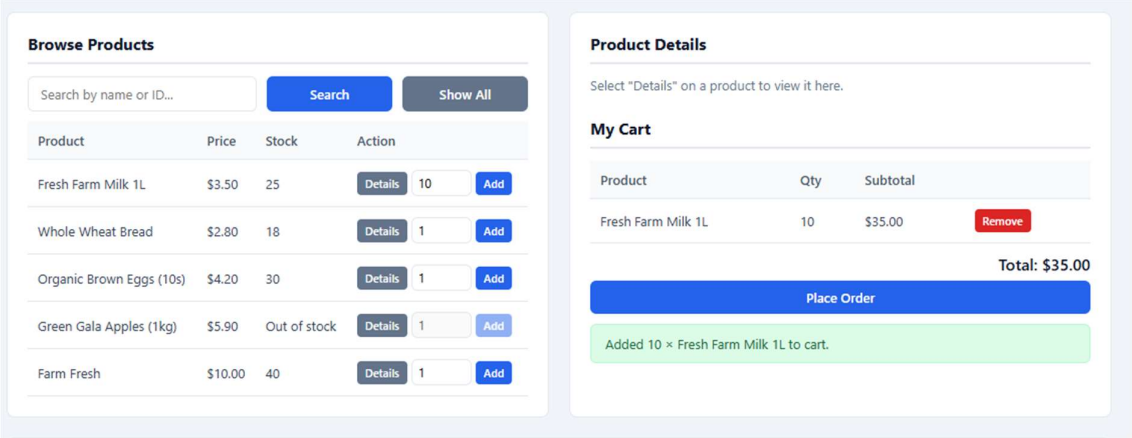


System shows up error message if the customer order more quantity that the quantity of stock

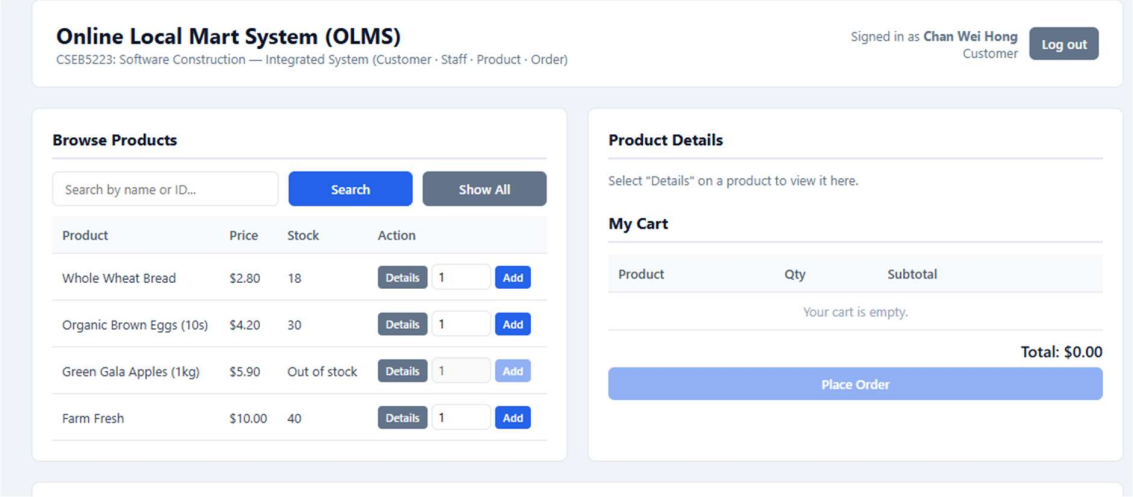


System searching functions

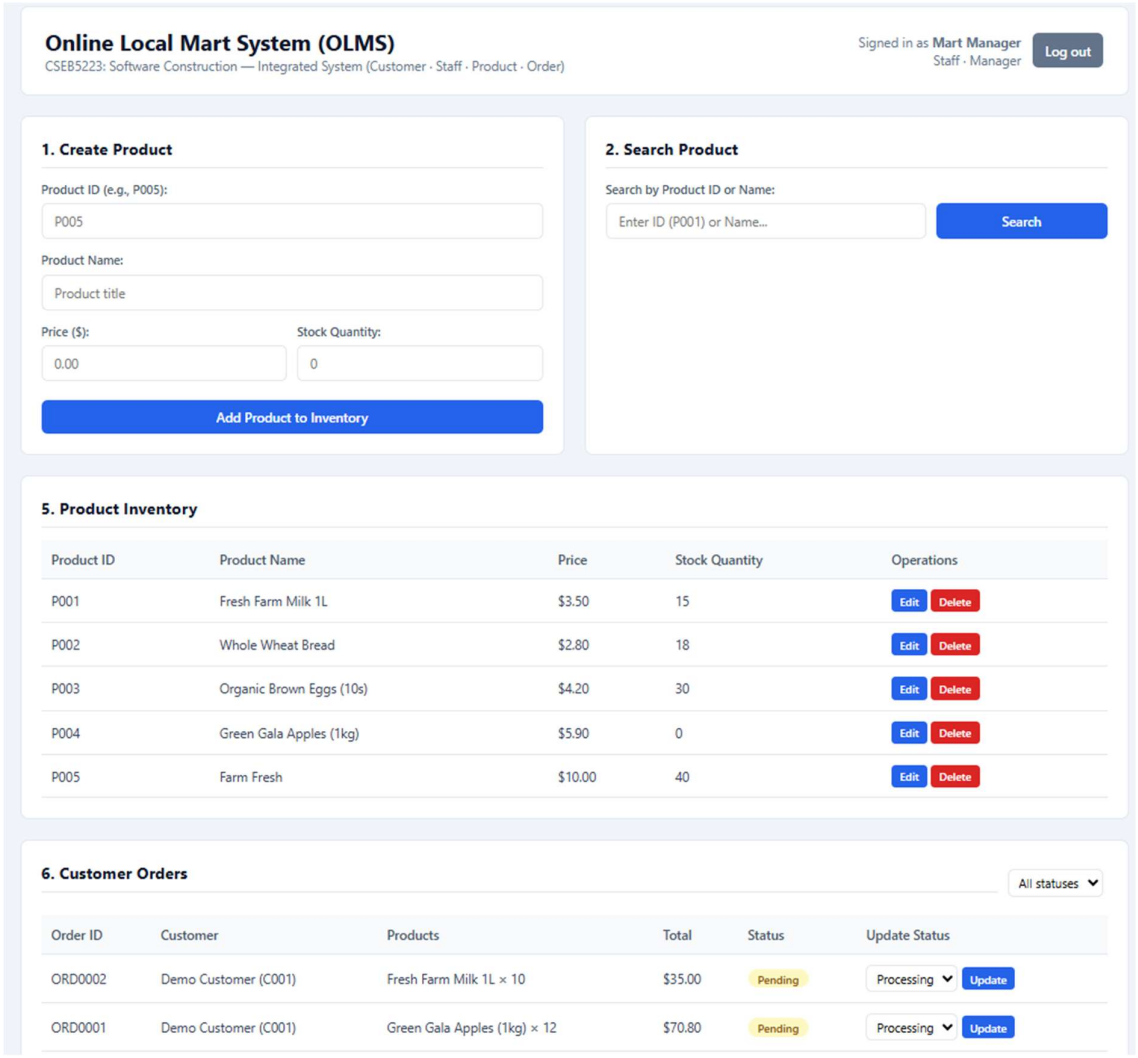




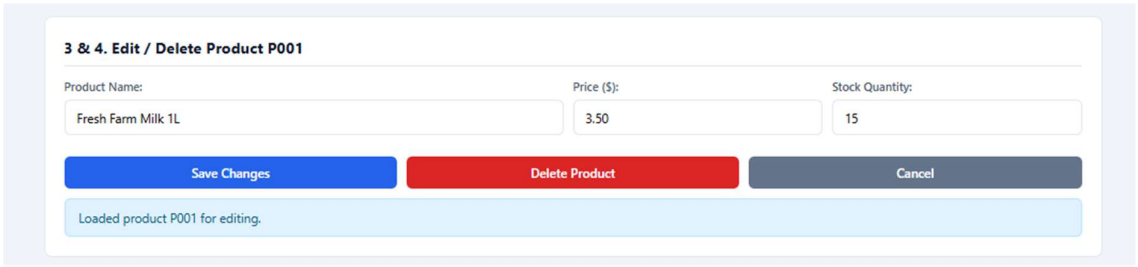
New customer can register throughout at the main page and directly logged in once they had registered.



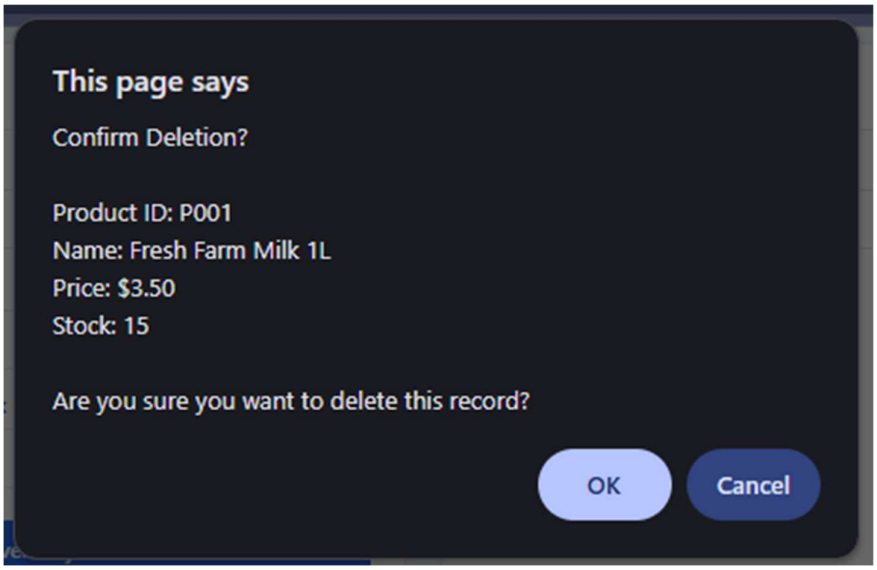
2. Admin (Staff) Page



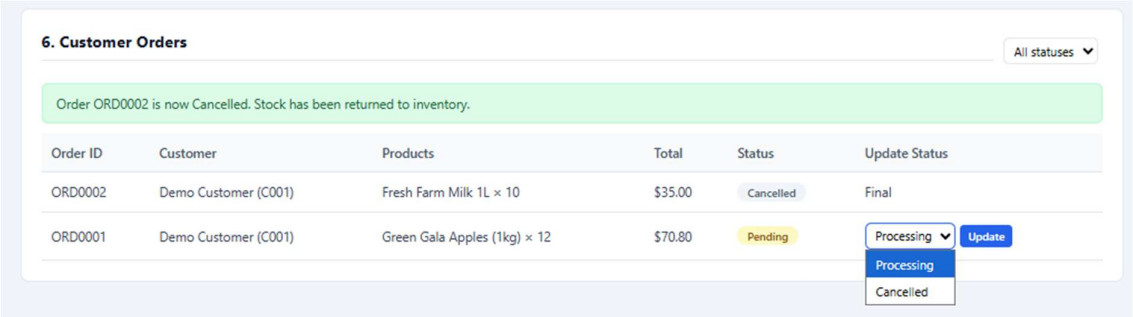
Staff can edit the product details with this edit button clicked



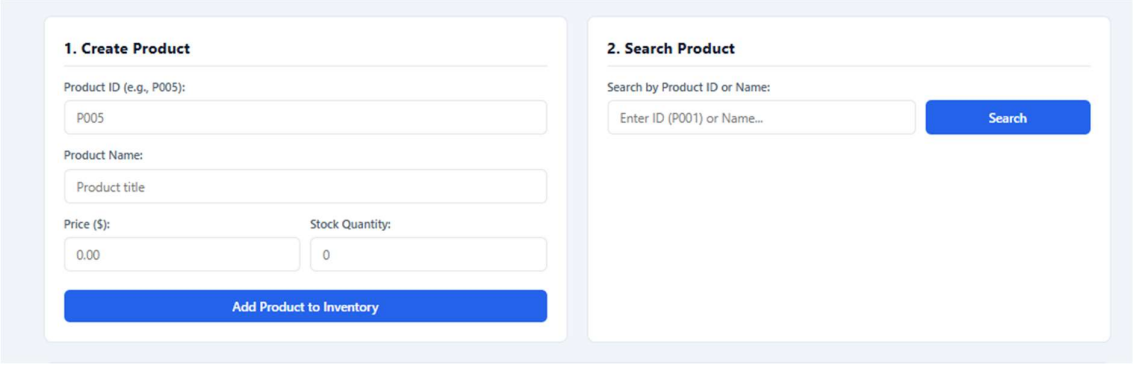
System will pop up a message for product deletion confirmation upon clicking Delete Product button



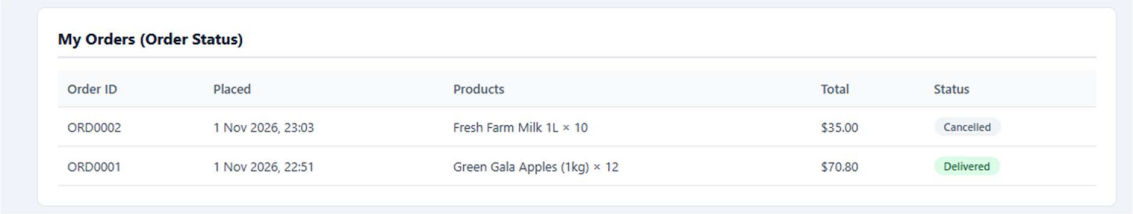
Staff can check customer's order and choose to accept or reject the orders



Staff can add new product and search for the product



Once the staff updated the status, the customer page can view up the latest status of their ordered product.



Activity 4: System Integration

Summary

Before this task, the repository contained only the Product module (the Lab 3 and Lab 4 Product CRUD page). The Customer, Staff and Order classes existed only in the UML class diagram and the documentation.

This task built the missing classes and connected all four UML classes – **Customer, Staff, Product and Order** – into one working web application. A customer can now register, log in, browse products, fill a cart and place an order. Staff can manage products and move orders through their status lifecycle. Stock is updated automatically whenever an order is placed or cancelled.

The integrated system was verified in two ways: 18 automated integration tests, and a complete end-to-end run of the real application as both a Customer and a Staff member. Every check passed.

1. What the task means

Integrate all classes or modules means connecting the separate parts of the system so they work together as one application, following the relationships shown in the UML class diagram.

Ensure the system works as intended means testing those connections, not only each class on its own. A complete real-world flow must work from start to finish, and the business rules must hold.

1.1 Starting point

Class	Before integration	After integration
Product	Working CRUD page (Lab 3 and 4)	Part of the Staff area; used by Customer and Order
Customer	UML diagram only	Implemented: register, login, browse, cart, place order, view order status
Staff	UML diagram only	Implemented: login, manage products, view orders, update order status
Order	UML diagram only	Implemented: create order, status lifecycle, cancel with restock

2. What was integrated

2.1 System structure

The code follows a layered MVC/Service design, with one file per layer. Each layer only uses the layers above it, so the files must load in this order: Classes → Repositories → Services → Functions.

File	Contents
index.html	The single web page for both roles: login and registration, the customer screens and the staff screens. Loads the four script files in order.
css/styles.css	All styling, including the order status badges and the phone layout.
js/Classes.js	Domain layer: OrderStatus, Validator, Product, Customer, Staff, Order.
js/Repositories.js	Data layer: ProductRepository, CustomerRepository, StaffRepository, OrderRepository (in-memory storage).
js/Services.js	Service layer: OrderService, AuthService, and createSystem(), which connects everything and loads the demo data.
js/Functions.js	UI layer: the functions behind every screen and button. Starts the app, so it loads last.
tests/IntegrationTests.html tests/IntegrationTests.js	The 18 automated integration tests. Open the HTML page in a browser to run them.
README.md	How to run the app and tests, the demo accounts and the UML method mapping.

2.2 Classes and their methods

Class	Methods	Purpose
Product	getDetails(), updateProduct(), reduceStock(), restoreStock()	A product with ID, name, price and stock quantity. Validates its own data.
Customer	register(), login(), browseProduct(), viewProductDetails(), addToCart(), removeFromCart(), getCart(), clearCart(), placeOrder(), viewOrderStatus()	A shopper who browses products and places orders.
Staff	login(), manageProduct(), viewOrder(), updateOrderStatus()	A store worker who manages products and customer orders.
Order	createOrder(), getItems(), getOrderStatus(), updateStatus(), getNextStatuses()	One placed order: its products, the prices paid, the total amount and the status.
OrderStatus	Pending, Processing, Shipped, Delivered, Cancelled	The allowed order statuses and the moves between them.
Validator	requireText(), email(), phone(), password()	Shared input checks used by Customer and Staff.

2.3 How the classes are connected

These are the links from the UML diagram, and how each one works in the integrated system.

Connection	What happens in the app
Customer → Product	The customer browses and searches products and opens a product's details.
Customer → Order	The customer places an order from the cart and sees the status of their own orders.
Order → Product	Placing an order reduces each product's stock. Cancelling an order returns the stock.
Staff → Product	Staff create, edit and delete products (the original Lab 3 and 4 module). Changes appear to customers straight away.
Staff → Order	Staff see every customer order, filter by status and update the status.

2.4 UML method to code mapping

UML method	Where it is implemented	Used by (screen)
Customer.register()	Customer.register()	Register form
Customer / Staff.login()	login(password) via AuthService.login()	Login form
Customer.browseProduct()	Customer.browseProduct()	Browse Products
Customer.viewProductDetails()	Customer.viewProductDetails() → Product.getDetails()	Details button
Customer.placeOrder()	Customer.placeOrder() → OrderService.placeOrder() → Order.createOrder()	Place Order
Customer.viewOrderStatus()	Customer.viewOrderStatus()	My Orders
Staff.manageProduct()	Staff.manageProduct(repo, action, data) with create, update or delete	Staff sections 1 to 5
Staff.viewOrder()	Staff.viewOrder()	Customer Orders
Staff.updateOrderStatus()	Staff.updateOrderStatus() → Order.updateStatus()	Update button
Product.updateProduct()	Product.updateProduct()	Save Changes
Order.getOrderStatus()	Order.getOrderStatus()	Status badges

2.5 Integration rules

- **Placing an order is all-or-nothing.** Every cart line is checked against stock before any stock is reduced, so a failed order never changes stock.
- **Orders keep the price paid.** Each order stores the unit price at the moment it was placed, so later price changes do not change old orders.
- **Order statuses only move forward:** Pending → Processing → Shipped → Delivered. An order can be cancelled only while it is Pending or Processing, and cancelling returns its stock.
- **Deleted products cannot be ordered.** The customer is told to remove the product from their cart.
- **Customers only see their own orders.** Staff see all orders.

2.6 Additions beyond the UML diagram

A few small additions were needed for the UML methods to work:

- A **cart** on Customer (addToCart, removeFromCart, getCart, clearCart), because placeOrder() needs a list of products to order.
- A private **password** field on Customer and Staff, because login() must check something. Passwords are kept in plain text, which is acceptable for an in-memory demo but not for a real system.
- A **Cancelled** status and Order.getNextStatuses(), so the screen only offers valid next statuses.
- A **product ID format** (P followed by 3 or more digits, such as P001) and prices limited to 2 decimal places.